

`<h1>` WEB APPLICATION `</h1>`

Introduction



`<p>` Er. Ramesh Sharma`</p>`



What is HTML

- HyperText Markup Language
- Structure of a web page

<> index.html > html

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Document</title>
7  </head>
8  <body>
9
10 </body>
11 </html>
```





HTML

- `<!DOCTYPE html>`

`<html>`: Root element

`<head>`: Metadata, title, styles

`<body>`: Content visible to users





HTML ELEMENTS Vs TAGS Vs ATTRIBUTES



```
8 <body>
9   <h1 class="title">This is title </h1>
10 </body>
```



HTML ELEMENTS

An element is the **complete structure**, including the **opening tag**, **content**, and **closing tag**.

Element: `<h1>This is title </h1>`

Content inside element: This is title





HTML TAGS

A tag marks the **start** or **end** of an HTML **element**.

`<h1>` → opening tag

`</h1>` → closing tag



HTML ATTRIBUTES

Attributes are extra information inside the opening tag that modify the **element's** behavior or appearance.

Element: `<h1 class='main-title' id='main_title'>This is title </h1>`

Attributes: class, id

Tags: `<h1></h1>`

Content inside element: This is title

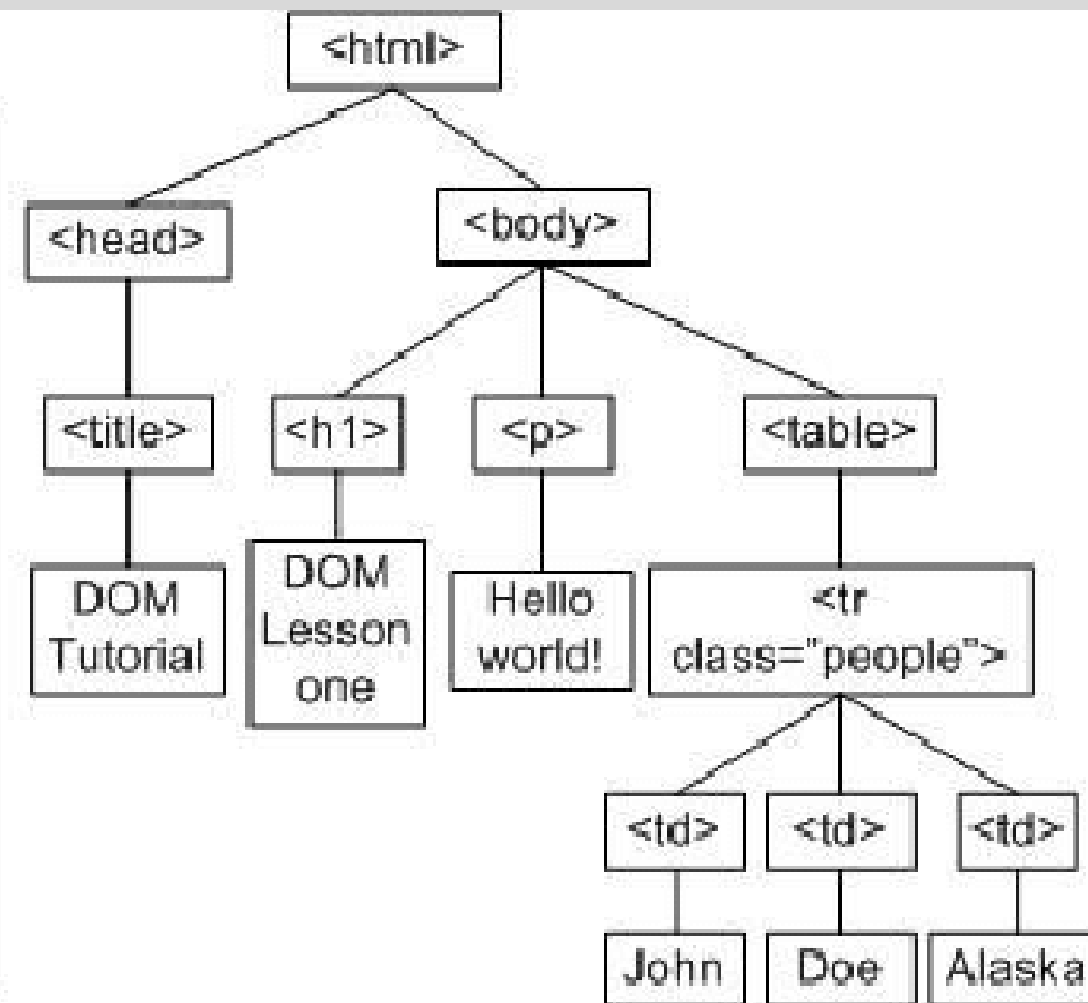




DOM TREE

```
<html>
  <head>
    <title>DOM Tutorial</title>
  </head>
  <body>
    <h1>DOM Lesson one</h1>
    <p>Hello world!</p>
    <table>
      <tr class="people">
        <td>John</td>
        <td>Doe</td>
        <td>Alaska</td>
      </tr>
    </table>
  </body>
</html>
```

test.html



DOM Tree



HTML HEADINGS

```
<!-- Headings -->  
<h1 class="title">This is title </h1>  
<h2>Heading 2</h2>  
<h3>Heading 3</h3>  
<h4>Heading 4</h4>  
<h5>Heading 5</h5>  
<h6>Heading 6</h6>
```

This is title

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6





HTML Paragraphs, Links and Images

```
<p>Paragraph</p>  
<a href="">Link 1</a>  

```

What are inline and block elements ?

Paragraph



[Link 1](#)





HTML Lists

```
<ul class="my-list">
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
  <li>Item 4</li>
</ul>
```

- Item 1
- Item 2
- Item 3
- Item 4

1. Home
2. About
3. Services
4. Contact

ul vs ol ?





HTML Tables

```
<table>
  <tr>
    <th> SN</th>
    <th> Section</th>
    <th> Room Id</th>
  </tr>
  <tr>
    <td>1</td>
    <td>A</td>
    <td>304</td>
  </tr>
  <tr>
    <td>2</td>
    <td>B</td>
    <td>305</td>
  </tr>
</table>
```

SN	Section	Room Id
1	A	304
2	B	305



HTML Forms

```
<!-- Forms -->  
<form action="#" method="post">  
  <label for="FullName ">Full Name: </label>  
  <input type="text">  
</form>
```

Username:

Email:





HTML Forms

```
<label>Gender:</label><br>
<input type="radio" name="gender"> Male
<input type="radio" name="gender"> Female<br><br>
<!-- Checkbox -->
<label>Skills:</label><br>
<input type="checkbox"> HTML
<input type="checkbox"> CSS
<input type="checkbox"> JavaScript<br><br>
```

Gender:

☐ Male ☐ Female

Skills:

☐ HTML ☐ CSS ☐ JavaScript

Department:

CSE ▾

CSE

IT

ECE





HTML Forms

```
<!-- Select Dropdown -->
<label>Department:</label><br>
<select>
  <option>CSE</option>
  <option>IT</option>
  <option>ECE</option>
</select><br><br>

<!-- Textarea -->
<label>Address:</label><br>
<textarea rows="4" cols="30"></textarea><br><br>
```

Department:

Address:





HTML Forms

```
<!-- Date -->
<label>Date of Birth:</label><br>
<input type="date"><br><br>

<!-- File Upload -->
<label>Upload Photo:</label><br>
<input type="file"><br><br>

<!-- Range -->
<label>Experience Level:</label><br>
<input type="range"><br><br>

<!-- Color -->
<label>Favorite Color:</label><br>
<input type="color"><br><br>
```

Date of Birth:



Upload Photo:

No file chosen

Experience Level:



Favorite Color:





HTML Forms

```
<!-- Hidden Field -->  
<input type="hidden" value="form123">  
  
<!-- Buttons -->  
<button type="submit">Submit</button>  
<button type="reset">Reset</button>
```

ACTION/POST/GET
?

Experience Level:



Favorite Color:



Submit

Reset





Semantic HTML5 Elements

Semantic HTML5 elements clearly describe their meaning and purpose to both the browser and the developer.(SEO)

```
<!-- Header -->
<header>
  <h1>My Website</h1>
</header>
<nav>
  <a href="#">Home</a> |
  <a href="#">About</a> |
  <a href="#">Contact</a>
</nav>
```

My Website

[Home](#) | [About](#) | [Contact](#)





Semantic HTML5 Elements

```
<main>
  <section>
    <h2>About Us</h2>
    <p>This is a section of content.</p>
  </section>

  <article>
    <h2>Latest News</h2>
    <p>This article can stand alone.</p>
  </article>
</main>
```

About Us

This is a section of content.

Latest News

This article can stand alone.





Semantic HTML5 Elements

```
<aside>
  <p>Related Links</p>
</aside>

<footer>
  <p>Copyright © 2026 My Website</p>
</footer>
```

Related Links

Copyright © 2026 My Website





CSS

CSS (Cascading Style Sheets) is used to **style** HTML elements and **control** layout, colors, fonts, spacing, etc.

```
p{  
  color:  red;  
  font-size: 16px;  
}
```





CSS Syntax

A CSS rule has three parts:

Selector – element to style

Property – what to change

Value – how to change it

```
h1 {  
  color: red; /* property: value */  
  font-size: 32px;  
}
```





Types of CSS

- Inline
- Internal
- External





Inline CSS

Directly in HTML tag using style attribute

```
<!-- Inline CSS -->  
<p style="color:  red;">Hello</p>
```





Internal CSS

Inside `<style>` tag in `<head>`

```
<style>
  p { color:  blue; }
</style>
```





External CSS

Separate `.css` file, linked with `<link>`

```
<link rel="stylesheet" href="style.css">
```





CSS Selectors

Selectors choose HTML elements to style.

- Element: `p {}`
- Id: `#main {}`
- Class: `.title {}`
- Group: `h1, h2 {}`
- Descendant: `div p {}`





CSS Selectors

```
<div>  
  <h1 class="about-us-title" id="about_us_title"> Who are we?</h1>  
  <p > Web <a href=""> Developers </a> </p>  
</div>
```





CSS Selectors

```
<div>
  <h1 class="about-us-title" id="about_us_title"> Who are we?</h1>
  <p > Web <a href=""> Developers </a> </p>
</div>
```

```
h1{
  color: ■ red;
}
#about_us_title{
  color: ■ beige;
}
.about-us-title{
  color: ■ blueviolet;
}
div h1 {
  color: ■ aqua;
}
h1, p{
  font-family: sans-serif;
}
```





CSS Selectors

```
<div>  
  <h1 class="about-us-title" id="about_us_title"> Who are we?</h1>  
  <p > Web <a href=""> Developers </a> </p>  
</div>
```

priority

```
#about_us_title {  
  color: ■ beige;  
}  
  
.about-us-title {  
  color: ■ blueviolet;  
}  
  
div h1 {  
  color: ■ aqua;  
}
```

```
h1{  
  color: ■ red;  
}  
#about_us_title{  
  color: ■ beige;  
}  
.about-us-title{  
  color: ■ blueviolet;  
}  
div h1 {  
  color: ■ aqua;  
}  
h1, p{  
  font-family: sans-serif;  
}
```

! important ?





CSS Box Model

- **Content** – text or images
- **Padding** – space inside the box
- **Border** – outline of the box
- **Margin** – space outside the box

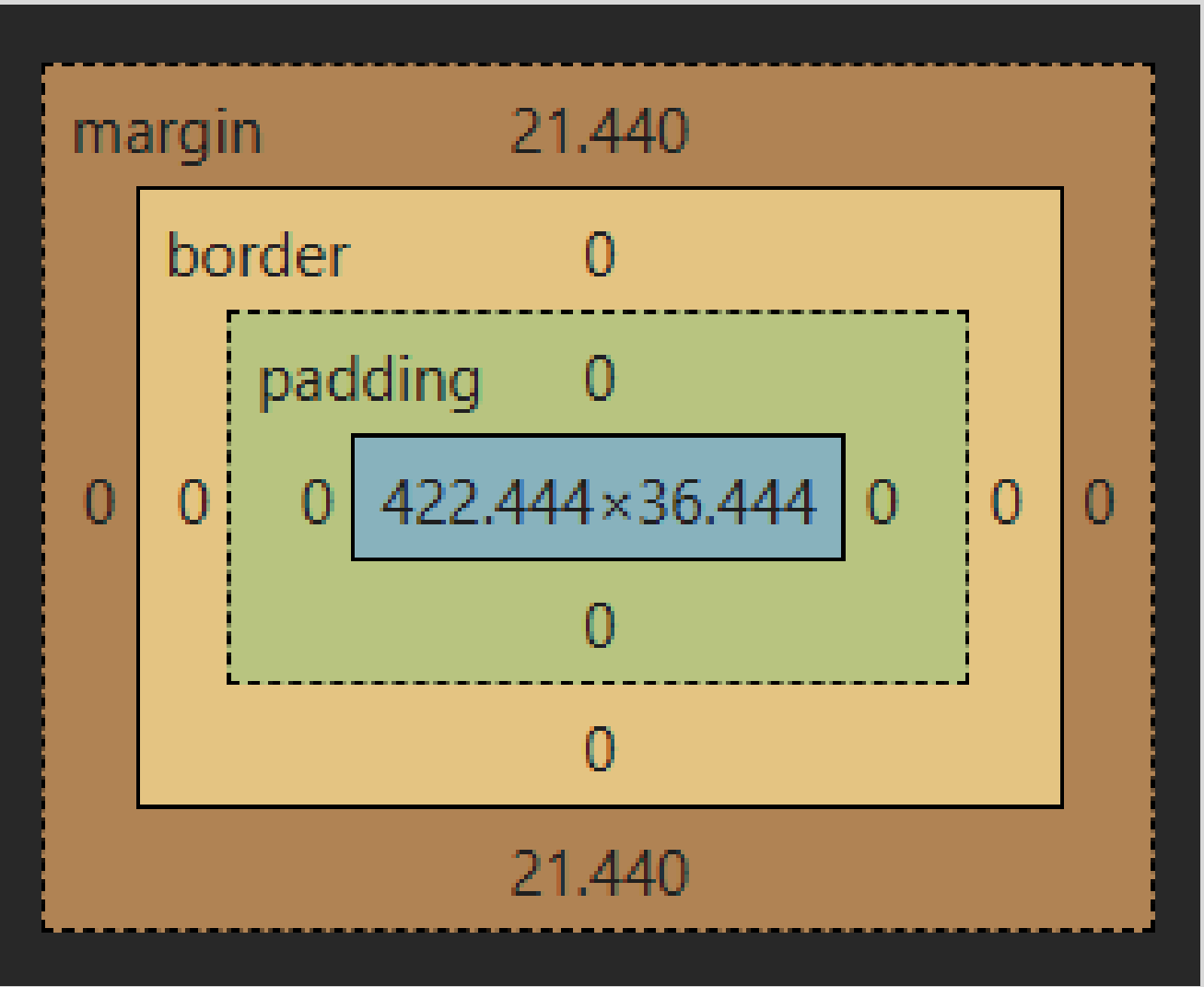




CSS Box Model

Who are we?

Web Developers





CSS Box Model

```
div{  
  padding: 5px;  
  margin: 10px;  
  border: 5px solid red;  
  /* Shorthand(width,style,color) */  
}
```

div

442.22 × 133.76

Color

■ #000000

Font 16px "Times New Roman"

Margin

10px

Padding

5px

ACCESSIBILITY

Name

Role

generic

Keyboard-focusable





CSS Background

background-color

Sets the background color of an element.

```
div {  
    background-color: lightblue;  
}
```





CSS Background

background-image

Adds an image as background.

```
div {  
    background-image: url("/assets/logo.webp");  
}
```





CSS Background

background-repeat

Controls tiling of background image.

```
background-repeat: repeat;  
background-repeat: no-repeat;  
background-repeat: repeat-x;  
background-repeat: repeat-y;
```





CSS Background

background-position

Sets the image position.

```
background-position: center;  
background-position: top right;  
background-position: 50% 50%;
```





CSS Background

background-size

Controls image scaling.

```
background-size: auto;  
background-size: cover;  
background-size: contain;  
background-size: 100px 200px;
```





CSS Background

background-attachment

Controls scrolling behavior.

```
background-attachment: scroll; /* default */  
background-attachment: fixed; /* parallax effect */
```





CSS Background

Gradient Backgrounds

```
background: linear-gradient(to right,  red,  yellow);
```

Shorthand

```
div {  
  background: url("image.jpg") no-repeat center cover  lightgray;
```





CSS Grid



PATRICIA BROWN-JOHNES

Dashboard

Typography

Grid

Tables

Forms

UI elements

Color schemes

Icons

Modals

Alerts

Accordion

Tabs

Buttons

Pages

Sharpen / UI elements / Icons

WEBSITE TRAFFIC

256,521



FILES DOWNLOADED

5,478,123



TOTAL SALES

14,789



TRAFFIC OVERVIEW

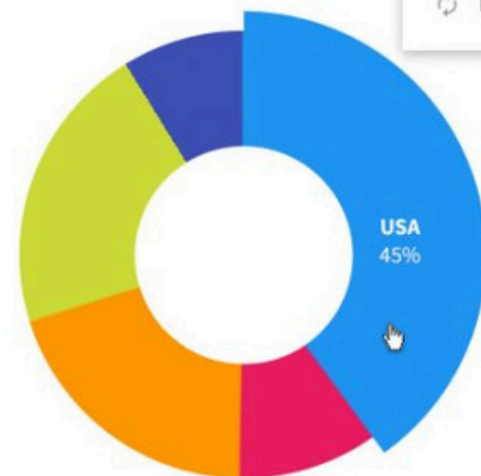
COUNTRIES

USED DEVICES

Time span

Chart type

Refresh



256,521

monthly total

10,907

today total

FREE STORAGE

77%

BANDWIDTH

1,5 Gb/s

RECENT TASKS

Lorem ipsum dlores amet voluptates quidem

Voluptates iusto quidem harem rerum

Nullam iusto ducimus 24/7

Lorem ipsum dlores amet voluptates quidem

Lorem ipsum dlores amet voluptates quidem

Lorem ipsum dlores amet voluptates quidem

SHARPEN

Followers

109,073

Subscribers

26,114

RECENT ORDERS

iPhone 6s + Apple Green cover IU/T

6 min. ago

MacBook Pro 15 "Retina CZ 2014

9 min. ago

SALES

2,547

Last month

4%

SUBSCRIBERS

32,547

Last month

4%



CSS Grid

Column1

Column2

Column3

Column4

Column5

Column6





CSS Grid

CSS Grid is a 2-dimensional layout system
→ Controls rows + columns at the same time.

Best for:

Page layouts, Dashboards, Cards
Complex UI structures





CSS Grid

```
.container {  
  display: grid;  
  grid-template-columns: auto auto;  
}
```

```
<div class="container">  
  <div>left</div>  
  <div>right</div>  
</div>
```

```
.container {  
  display: grid;  
  grid-template-rows: auto auto;  
}
```





CSS Grid

```
<div class="container">  
  <div>left</div>  
  <div>right</div>  
</div>
```

```
.container {  
  display: grid;  
  grid-template-columns: auto auto;  
  grid-template-columns: repeat(3, 1fr);  
}
```





CSS Grid: Named Grid Areas

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar content"  
    "footer footer";  
  grid-template-columns: 1fr 3fr;  
}
```

```
.header {  
  grid-area: header;  
  background-color: #aliceblue;  
}  
.sidebar {  
  grid-area: sidebar;  
  background-color: #antiquewhite;  
}  
.content {  
  grid-area: content;  
  background-color: #blue;  
}  
.footer {  
  grid-area: footer;  
  background-color: #green;  
}
```





Alignment (Container)

```
.container {  
  display: grid;  
  justify-items: center; /* horizontal */  
  align-items: center; /* vertical */  
  
  justify-content: space-between;  
  align-content: center;  
}
```





CSS Display Types

The display property defines how an element is displayed in the layout.

INLINE

Stays in line, width/height ignored

BLOCK

Breaks line, takes full width

FLEX

Container aligns children, flexible layout





CSS Display Types

```
.inline-img {  
  display: inline;  
  border: 2px solid  red;  
}  
  
.block-img {  
  display: block;  
  border: 2px solid  green;  
}
```



Inline

Block Image:



Block

```
<p>Inline Image:</p>  
  
  
<p>Block Image:</p>  

```





CSS Display Types

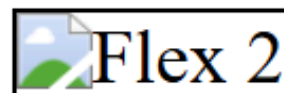
```
.flex-container {  
  display: flex;  
  justify-content: center;  
  gap: 20px;  
  border: 2px solid blue;  
  padding: 10px;  
}
```

```
<p>Flex Container:</p>  
<div class="flex-container">  
    
    
    
</div>
```

Flex Container:



Flex 1



Flex 2



Flex 3





CSS Positioning

The position property defines how an **element is positioned** in the **layout**.

Position	Behavior
static	Default, normal flow
relative	Moves relative to its normal position
absolute	Positioned relative to nearest ancestor with position other than static
fixed	Stays fixed in viewport, does not move on scroll





CSS Positioning

Positioning Example

Static Box

```
<div class="static-box">Static Box</div>
```

```
.static-box {  
  position: static;  
  background-color: #f0f0f0;  
  padding: 10px;  
}
```



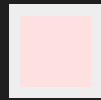


Positioning Example

Relative Box

CSS Positioning

```
<div class="relative-box">Relative Box</div>

.relative-box {
  position: relative;
  top: 10px;
  left: 20px;
  background-color:  #ffe0e0;
  padding: 10px;
}
```





Positioning Example

Absolute Box

CSS Positioning

```
<div class="absolute-box">Absolute Box</div>

.absolute-box {
  position: absolute;
  top: 50px;
  left: 50px;
  background-color: #e0ffe0;
  padding: 10px;
}
```





Positioning Example

CSS Positioning

```
<div class="fixed-box">Fixed Box</div>

.fixed-box {
  position: fixed;
  bottom: 10px;
  right: 10px;
  background-color: #e0e0ff;
  padding: 10px;
}
```

Fixed Box





Font Properties

```
p {  
  font-family: Arial;  
  font-size: 18px;  
  font-weight: bold;  
  font-style: italic;  
}
```

Controls style, size, and thickness





Text properties

```
p {  
  text-align: center;  
  text-decoration: underline;  
  text-transform: uppercase;  
}
```

Controls alignment, decoration, and transformation





Bootstrap

Bootstrap is a CSS framework

Helps create responsive and mobile-first websites



Get started with Bootstrap

Bootstrap is a powerful, feature-packed frontend toolkit. Build anything—from prototype to production—in minutes.

 [getbootstrap/](#)





Bootstrap CSS

```
<!-- Bootstrap CSS -->
```

```
<link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
```

```
rel="stylesheet"
```

```
/>
```

Layout System, Typography (Text Styling), Colors & Backgrounds,
Spacing (Margin & Padding), Buttons, Forms & Inputs
Navigation (Menus)





Bootstrap JS

```
<!-- Bootstrap JS -->  
<script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.8/dist/umd/popper.min.js">  
  
</script>  
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.min.js">
```

```
    <!-- Bootstrap JS -->  
    <script src="https://cdn.jsdelivr.net/npm/@popperjs/core@2.11.8/dist/umd/popper.min.js">  
  
        </script>  
        <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.min.js">  
  
            </script>
```





Dropdowns

Menu ▼

Home

About

Contact





Modal

Launch demo modal

Modal title



...

Close

Save changes





Modal

Launch demo modal

Modal title



...

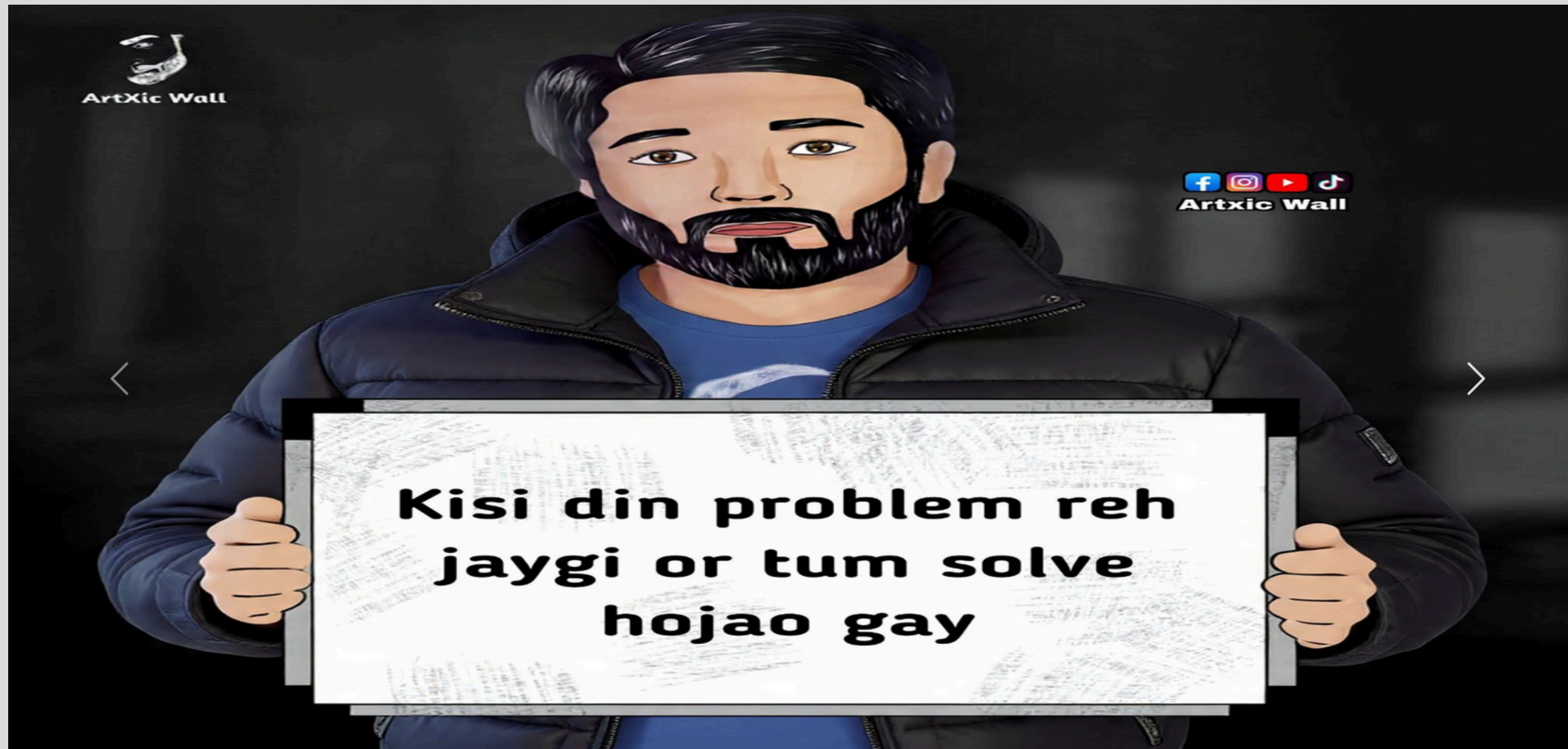
Close

Save changes





Carousel(Sliders)





Card



Me and bro discussing problems with the education system instead of actually studying

Card title

Some quick example text to build on the card title and make up the bulk of the card's content.

Go somewhere



Card title

Some quick example text to build on the card title and make up the bulk of the card's content.

Go somewhere



Card title

Some quick example text to build on the card title and make up the bulk of the card's content.

Go somewhere



Pagination

Previous

1

2

3

Next





Accordion

Accordion Item #1



This is the first item's accordion body. It is shown by default, until the collapse plugin adds the appropriate classes that we use to style each element. These classes control the overall appearance, as well as the showing and hiding via CSS transitions. You can modify any of this with custom CSS or overriding our default variables. It's also worth noting that just about any HTML can go within the `.accordion-body`, though the transition does limit overflow.

Accordion Item #2



Accordion Item #3





Other Coponents

Alerts

Badge

Breadcrumb

Buttons

Button group

Card

Carousel

Close button

Collapse

Dropdowns

List group

Navbar

Navs & tabs

Offcanvas

Pagination

Placeholders

Popovers

Progress

Scrollspy

Spinners

Toasts

Tooltips



Javascript

- JavaScript is a **programming language** for the web
- Makes web pages **interactive** and **dynamic**
- Runs in the **browser** (and also on server with Node.js)





Internal vs External

```
<body>
  <!-- Internal JS -->
  <script>
    const a = 10; // number
  </script>
  <!-- External JS -->
  <script src="script.js"></script>
</body>
```





Data Types

Primitive data types store **single, simple** values and are **immutable**.
Examples: number, string, Boolean, Undefined, Null, BigInt, Symbol

Non-primitive data types store **multiple** values or **complex** data and
are **mutable**. **Examples:** Object, Array, Function

Primitive data types: copied by value,
non-primitive data types: copied by reference.





Data Types

```
<body>
  <script>
    // Primitive Data Types
    let a = 10; // number
    const user = "john Doe"; //string
    const isActive = true; //boolean

    //non-primitive
    const array1 = [4, 5, 6, 7]; //array
    const object1 = {
      name: "ramesh",
      age: 18,
      active: true,
    };
  </script>
</body>
```



Variables

A variable is a container used to store data so it can be reused and changed later.

var (outdated)

- Function-scoped
- Can be redeclared
- Can be updated

Ramesh Sharma

let

- Block-scoped
- Cannot be redeclared
- Can be updated

Ramesh Sharma

const

- Block-scoped
- Cannot be redeclared
- Cannot be updated

Ramesh Sharma





Variables

var (outdated)

```
var a = 39;  
var a = 29; → Yes  
a = 33; → Yes
```

Ramesh Sharma

let

```
let a = 39;  
let a = 33; → NO  
a = 29; → Yes
```

Ramesh Sharma

const

```
const a = 39;  
const a = 28; → NO  
a = 33 → NO
```

Ramesh Sharma





Variables

Global Scope

```
let a = 10;

function test() {
  console.log(a);
}
```

Function Scope

```
function test() {
  let a = 10;
  console.log(a);
}
```

Block Scope

```
if (true) {
  let x = 5;
}
```





Hoisting with var

Hoisting is JavaScript's behavior of moving declarations to the top of their scope. Only declarations are hoisted, not initializations.

```
console.log(a); // undefined
```

```
var a = 5;
```

```
console.log(a); // 5
```

```
var a; // declaration is hoisted to top
```

```
console.log(a); // undefined
```

```
a = 5; // initialization stays in place
```

```
console.log(a); // 5
```





Hoisting with let and const

```
console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;

console.log(c); // ReferenceError
const c = 20;
```

let and const are hoisted, but they are in a temporal dead zone (TDZ)

Temporal Dead Zone is the time between the start of a block (or scope) and the point where a let or const variable is declared





Hoisting with Functions

```
greet(); // "Hello!"  
  
function greet() {  
    console.log("Hello!");  
}
```





Operators

Arithmetic: + - * / % ** ++ -- | **Assignment:** = += -= *= /= %=

| **Comparison:** == === != !== > < >= <= | **Logical:** && || ! |

String: + += | **Conditional (Ternary):** ? : | **Type:** typeof

instanceof | **Bitwise:** & | ^ ~ << >> >>>

loose equality(value) and **strict equality(value, type)**

(5 == '5' , true)

(5 === '5', false)





Conditional Statements(if-else)

```
let marks = 50;

if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

```
let marks = 75;

if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 75) {
  console.log("Grade B");
} else if (marks >= 50) {
  console.log("Grade C");
} else {
  console.log("Fail");
}
```

```
let age = 20;
let canVote = age >= 18 ? "Yes" : "No";
console.log(canVote);
```





Conditional Statements(switch)

```
let day = 3;
switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Another day");
}
```





Loops

```
for (initialization; condition; increment / decrement) {  
    // code block  
}  
for (let i = 0; i < 5; i++) {  
    console.log(i);  
}
```

```
let i = 0;  
while (i < 5) {  
    console.log(i);  
    i++;  
}  
do {  
    console.log(i);  
    i++;  
} while (i < 5);
```





Loops(for-in , for-of)

```
let person = { name: "Ramesh", age: 25 };

for (let key in person) {
  console.log(key + ": " + person[key]);
}

let fruits = ["Apple", "Banana", "Mango"];

for (let fruit of fruits) {
  console.log(fruit);
}
```





Loop Control Statements(break/continue)

```
for (let i = 0; i < 5; i++) {  
    if (i === 3) break; // stops loop at 3  
    console.log(i);  
}  
  
for (let i = 0; i < 5; i++) {  
    if (i === 3) continue; // skips 3  
    console.log(i);  
}
```





Loop Control Statements(break/continue)

```
for (let i = 0; i < 5; i++) {  
    if (i === 3) break; // stops loop at 3  
    console.log(i);  
}  
  
for (let i = 0; i < 5; i++) {  
    if (i === 3) continue; // skips 3  
    console.log(i);  
}
```





Functions- Declaration (Named Function)

```
function functionName(parameters) {  
    // code block  
}  
functionName()  
  
function greet(name) {  
    console.log("Hello, " + name + "!");  
}  
greet("John"); // Output: Hello, John!
```





Function Expression

```
const greet = function (name) {  
  console.log("Hello, " + name + "!");  
};  
  
greet("John"); // Output: Hello, John!
```





Arrow Function [ES6]

```
const add = (a, b) => a + b;
```

```
console.log(add(5, 3)); // Output: 8
```

```
const multiply = (a, b) => {  
  let result = a * b;  
  return result;  
};
```





Return Statement

```
function square(x) {  
  return x * x;  
}
```

```
console.log(square(5)); // 25
```





Parameters and Arguments

```
function sum() {  
  let total = 0;  
  for (let i of arguments) {  
    total += i;  
  }  
  return total;  
}  
  
console.log(sum(1, 2, 3, 4)); // 10
```





Parameters and Arguments

```
function greet(name = "Guest") {  
    // default parameter  
    console.log("Hello, " + name);  
}
```

```
greet(); // Hello, Guest
```

```
greet("Balén"); // Hello, balen
```





DOM Manipulation

DOM (Document Object Model) is a **tree-like structure** created by the **browser** that represents an **HTML document**.

JavaScript uses the DOM to **access, modify, add, or remove elements** on a web page **dynamically**.

Ramesh Sharma





DOM Manipulation

Why DOM Manipulation?

- Update UI without reloading the page
- Handle user interactions (click, input, submit)
- Create dynamic and interactive web apps

Ramesh Sharma





DOM - Selecting Elements

getElementById: document.getElementById("myDiv")

getElementsByClassName:

document.getElementsByClassName("myClass")

getElementsByTagName: document.getElementsByTagName("p")

querySelector: document.querySelector(".myClass")

querySelectorAll: document.querySelectorAll(".myClass")

Ramesh Sharma





DOM - Selecting Elements

Creating & Removing Elements:

```
const div =  
document.createElement("div");  
parent.appendChild(div);
```

```
element.remove();
```

Ramesh Sharma

Changing Styles:

```
element.classList.add("active");  
element.classList.remove("active"  
);
```

Ramesh Sharma

Changing Content:

```
element.textContent = "Hello"
```

Ramesh Sharma





Events & Event Listener

An Event Listener allows JavaScript to **listen for user actions (events)** and execute code when they occur.

```
element.addEventListener("event", callback);
```

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

Ramesh Sharma





Common Events

click – mouse click

input – typing in input field

change – value change (after focus out)

submit – form submission

mouseover – hover

scroll – page scroll

keydown – key press

ramesharma2025/js-
modal-form



Ramesh Sharma





Form Handling and Validation

Form handling means: **getting values from input fields** responding to user **actions** (submit, input, change) processing the data (**validate, store, send**).

Ramesh Sharma





Form Handling and Validation

Validation means **checking** user input before **accepting it**.

Examples:

username must contain @ - **//.includes("@")**

password must be at least 8 characters - **// .length < 8**

email should be valid - **//regex.test()**

fields should not be empty - **//.trim() === ""**

rameshsharma2025/js-
modal-form



Ramesh Sharma





Regular Expression [Regex]

- A Regular Expression is a **pattern** used to **search, match, validate, or replace text**.
- **Smart filter for strings**

Ramesh Sharma

Example

```
const regex = /^(?=.*[A-Za-z])(?=.*\d)(?=.*@){8,}$;/
```

```
const regex = /@/;  
const regex = /^(?=.*[A-Za-z])/;  
const regex = /^(?=.*@)/
```

Ramesh Sharma





Local Storage & Session Storage

Local Storage lets the browser “remember” things even after **closing the tab or browser.**

Ramesh Sharma

Session Storage keeps **temporary** data only while the **tab is open.**

Ramesh Sharma





Local Storage & Session Storage

STORE

```
localStorage.setItem("usr", username);  
  
localStorage.setItem("pwd",  
password);  
  
localStorage.setItem("userData",  
JSON.stringify(data));
```

Ramesh Sharma

RETRIEVE

```
localStorage.getItem("usr");  
localStorage.getItem("password");  
  
JSON.parse(localStorage.getItem("  
userData"));
```

Ramesh Sharma





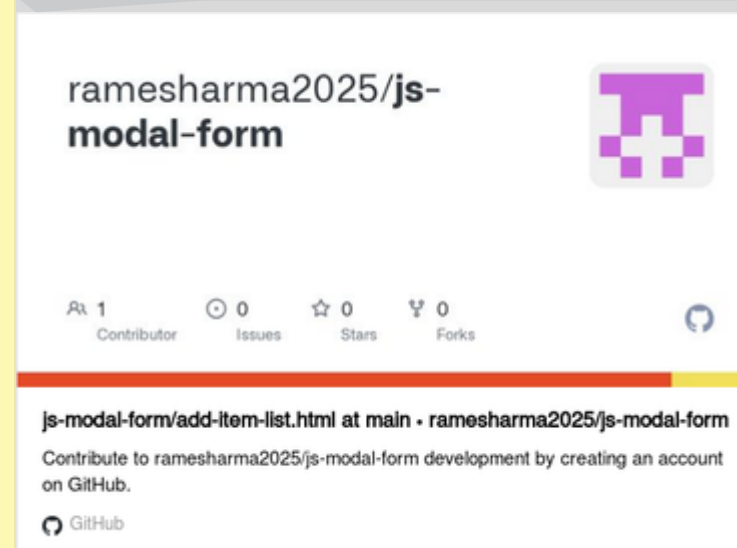
GUI Interactions (Dynamic UI)

- **Clicking** a button to show/hide content.
- **Typing** in a form and seeing validation messages.
- **Updating** a table or list in real-time.
- **Changing** colors, images, or styles on hover/click.

Ramesh Sharma

DOM Manipulation, Eventlistener, Live Form Validation, Dynamic UI

Ramesh Sharma





JavaScript Library: jQuery

jQuery is a **JavaScript** library created to simplify **HTML DOM manipulation, event handling, animations, and AJAX**.

It's **cross-browser compatible**, which was a big deal before modern JS standards.

Ramesh Sharma

DOM shortcuts (selecting and manipulating elements).

Event handling (clicks, input events).

AJAX basics (fetching data dynamically)

Ramesh Sharma





JavaScript Library: jQuery - DEMO

ramesharma2025/**js-**
modal-form





Modern JavaScript (ES6+) E

ES6 (ECMAScript 2015) introduced **cleaner syntax**, **better readability**, and **powerful features** that make JavaScript easier and safer to write.

Ramesh Sharma





[ES6+] - Variable Declarations

let → Block-scoped, re-assignable

const → Block-scoped, constant reference

Safer than var, avoids hoisting issues

Ramesh Sharma





[ES6+] - Arrow Functions

An arrow function is a **shorter and cleaner** way to write functions in JavaScript.

Ramesh Sharma

```
const functionName = () => { // code};
```

```
const add = (a, b) => a + b; //arrow
```

```
function add(a, b){  
    return a+b  
}
```

Ramesh Sharma





[ES6+] - Arrow Functions

When to Use Arrow Functions

- Callbacks
- Array methods (map, filter, reduce)
- Short utility functions

Ramesh Sharma





[ES6+] - Template literals

Template literals are a modern way to create strings in JavaScript using backticks ` instead of quotes.

Ramesh Sharma

OLD: `console.log("My name is " + name);`

NEW: `console.log(`My name is ${name}`);`

`console.log(`Sum is ${a + b}`);`

Ramesh Sharma





[ES6+] - Destructuring

Destructuring allows you to extract values from arrays or objects and store them in variables in a single line.

Ramesh Sharma

Array-OLD

```
const numbers = [10, 20, 30];  
let a = numbers[0];  
let b = numbers[1];
```

Ramesh Sharma

Array NEW

```
const numbers = [10, 20, 30];  
const [a, b] = numbers;  
  
console.log(a, b); // 10 20
```

Ramesh Sharma





[ES6+] - Destructuring

```
function display({ name, age }) {  
  console.log(name, age);  
}
```

Ramesh Sharma

Object-OLD

```
const user = { name: "John", age: 22 };  
let name = user.name;  
let age = user.age;;
```

Ramesh Sharma

Object-NEW

```
const user = { name: "John", age: 22 };  
  
const { name, age } = user;
```

Ramesh Sharma





[ES6+] - Spread Operator [...]

Spreads (expands) values

Uses:

Copy arrays/objects

Merge arrays/objects

Pass values to functions

Ramesh Sharma

Example - Array

```
const nums = [1, 2, 3];  
const newNums = [...nums, 4, 5];
```

```
console.log(newNums); // [1,2,3,4,5]
```

Ramesh Sharma

```
const user = { name: "John", age: 22 };  
const newUser = { ...user, role: "Admin" }
```

Ramesh Sharma





[ES6+] - Rest Operator [...]

Collects values into an array

Used in:

Function parameters

Destructuring

Ramesh Sharma

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b);  
}  
sum(1, 2, 3, 4); // 10
```

Ramesh Sharma

```
const [first, ...others] = [1, 2, 3, 4];  
console.log(others); // [2,3,4]
```

Ramesh Sharma

```
const user = { name: "John", age: 22, city:  
  "Kathmandu" };  
const { name, ...details } = user;
```

Ramesh Sharma





[ES6+] - Array Loops

forEach()

Purpose: Perform an action on each element

Returns: Nothing (undefined)

```
users.forEach(user => {  
  console.log(user.name);  
});
```

Ramesh Sharma

map()

Purpose: Transform array elements

Returns: New array

```
const names = users.map(user =>  
  user.name);
```

Ramesh Sharma





[ES6+] - Array Loops

filter()

Purpose: Select elements based on condition

Returns: New array

```
const adults = users.filter(user =>
user.age >= 18);
```

Ramesh Sharma

reduce()

Purpose: Reduce array to a single value.

Returns: Any value (number, object, array)

```
const total = prices.reduce((sum,
price) => sum + price, 0);
```

Ramesh Sharma





[ES6+] - Array Loops

find()

Purpose: Find first matching element.

Returns: Single element or undefined

```
const user = users.find(u => u.id === 5);
```

Ramesh Sharma

//Nested Loop

```
users  
  .filter(u => u.active)  
  .map(u => u.name).  
  .find(u => u.name === "John");
```

Ramesh Sharma





[ES6+] - Modules & Imports

export:

```
export const add = (a, b) => a + b;
```

```
export default function greet() {  
  console.log("Hello");  
}
```

Ramesh Sharma

import:

```
import { add } from "./math.js";
```

```
import greet from "./greet.js";
```

Ramesh Sharma





Sync Operations: Callbacks, Promises, Async/Await

Synchronous Operations:

- Code runs line by line
- Each task blocks the next task
- Slow tasks freeze the UI

Ramesh Sharma

Example:

```
console.log("Start");  
  
alert("Blocking operation");  
  
console.log("End");
```

Ramesh Sharma





Sync Operations: Callbacks, Promises, Async/Await

Asynchronous Operations:

- Long tasks run in the background
- Main thread stays free
- UI remains responsive

Ramesh Sharma

Common Async Tasks

- API calls
- File reading
- Timers (setTimeout)
- Database operations

Ramesh Sharma





Callbacks

A **callback** is a **function** **passed as an argument** to another function, executed after a task completes.

Why Callbacks?

- Handle async results.
- Execute code after completion.

Example:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data received");  
  }, 2000);  
}
```

Ramesh Sharma



Callbacks- Callback Hell (Problem)

Nested callbacks become hard to read & maintain.

- Difficult to debug
- Poor readability

Ramesh Sharma

Example:

```
getUser(id, () => {  
  getOrders(() => {  
    getPayment(() => {  
      //Code here  
    });  
  });  
});
```

Ramesh Sharma





Promises

A **Promise** represents a value that will be:

- **Fulfilled**
- **Rejected**
- **Pending**

Promise States:

Pending → Fulfilled / Rejected

Ramesh Sharma

Creating a Promise:

```
const fetchData = new  
Promise((resolve, reject) => {  
  setTimeout(() => {  
    resolve("Data fetched");  
  }, 2000);  
});
```

Ramesh Sharma





Promises

Consuming a Promise:

```
fetchData
  .then(result =>
    console.log(result))
  .catch(error =>
    console.log(error));
```

Ramesh Sharma

Promise Chaining (Solution to Callback Hell):

```
getUser(id)
  .then(user => getOrders(user))
  .then(orders => getPayment(orders))
  .then(payment =>
    console.log("Completed"))
  .catch(err => console.error(err));
```

Ramesh Sharma





Async / Await

- A modern syntax built on top of Promises.
- Makes async code look synchronous.

async Keyword

Makes a function return a Promise

Ramesh Sharma

```
async function fetchData() {  
  return "Hello";  
}
```

await Keyword

- Pauses execution until Promise resolves.
- Can only be used inside async functions.

Ramesh Sharma





Async / Await

```
async function processData() {  
  try {  
  
    const user = await getUser(id);  
    const orders = await getOrders(user);  
    const payment = await getPayment(orders);  
    console.log("Completed");  
  
  } catch (error) {  
    console.log(error);  
  }  
}
```

- Most readable
- Easier debugging
- Industry standard
- **returns a Promise**
- **Promise.resolve()**
- **Promise.reject()**

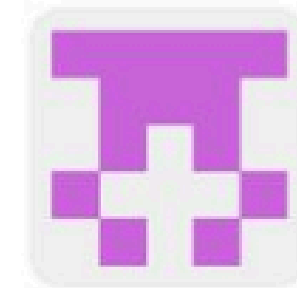
Ramesh Sharma

Ramesh Sharma



Callbacks, Promises, Async/Await: Example

ramesharma2025/**js-modal-form**



1

Contributor



0

Issues



0

Stars



0

Forks



js-modal-form/js-callback-async.html at main • ramesharma2025/js-modal-form

Contribute to ramesharma2025/js-modal-form development by creating an account on GitHub.

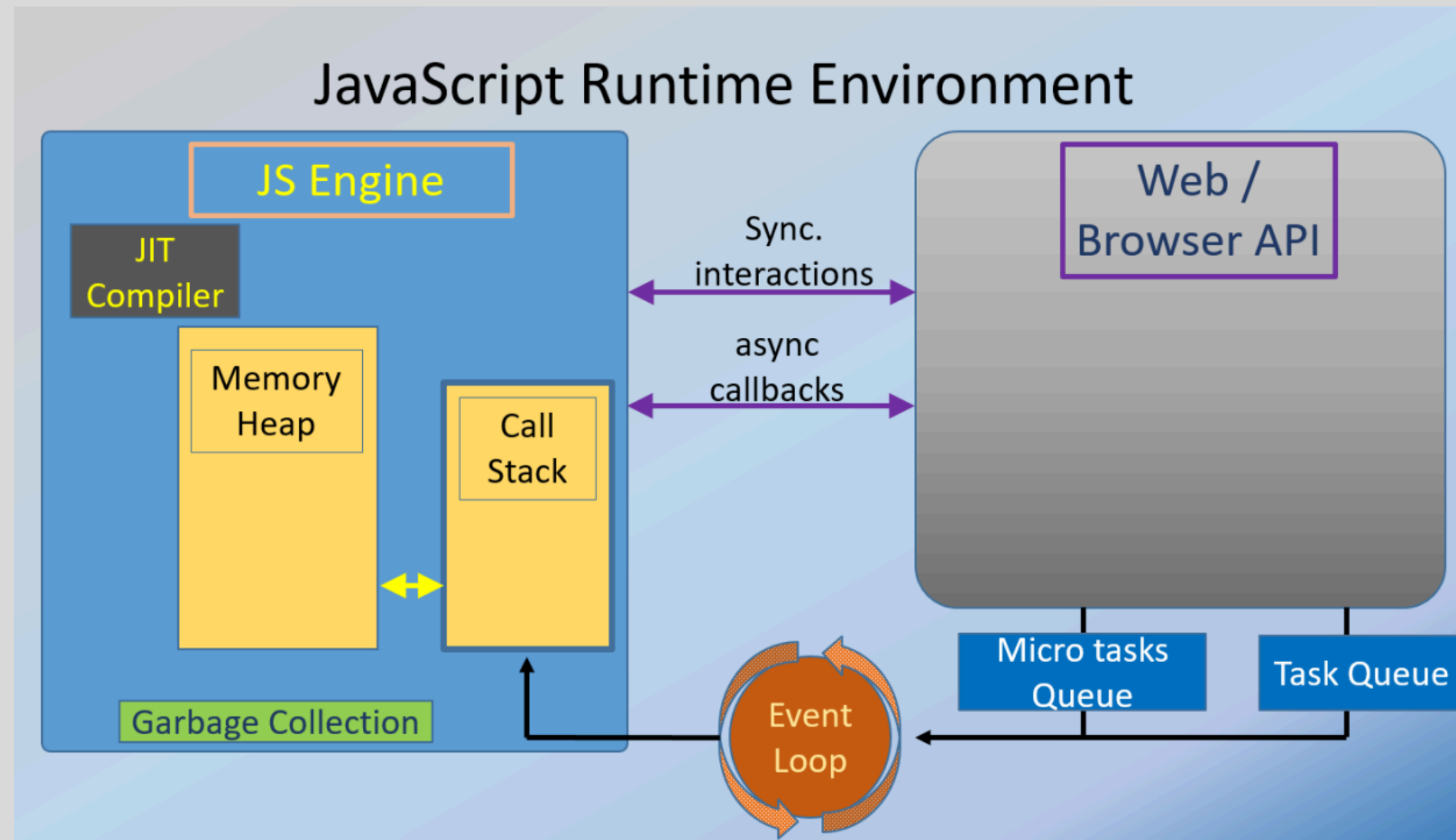


GitHub





Javascript Runtime Envionment



```
<h1 class="thank-you" style="color:#000;  
background-color:orange;">
```

THANK YOU

```
</h1>
```

